1.假设我们来用 LRU 作为缓冲区置换策略，当我们向 Buffer Manager 发出一个读页请求时，请讨论一下： （1）如果页不在緩冲区中，我们需要从磁盘中读入该页。请问如何才能在缓冲区不满的时候快速地返回一个 free 的 frame？请给出至少两种策略，并分析一下各自的时间复杂度。 （2） 如何才能快速地判断所请求的页是否在缓冲区中？如果请求的页在缓冲区中，如何快速返回该页对应的 frame 地址？请给出至少两种策略，并分析一下各自的时间复杂度。

（1）

**策略 1：维护一个空闲 frame 列表**

当缓冲区初始化时，可以将所有的 frame 预先标记为“空闲”。同时维护一个链表或栈，存储所有空闲的 frame。当需要分配一个新的 frame 时，只需从链表或栈顶取出一个空闲的 frame。

**时间复杂度：**O(1)
因为从链表或栈中取出空闲 frame 的操作只需要常数时间，无需遍历缓冲区。

**策略 2：使用位图（Bitmap）**

可以为缓冲区中的每个 frame 维护一个二进制位（bit），用来表示该 frame 是空闲还是已被使用。当需要找到一个空闲 frame 时，可以扫描位图，找到第一个标记为“空闲”的 frame。

**时间复杂度：**O(n)，n 是缓冲区的 frame 数量。
需要在最坏情况下扫描整个位图来寻找空闲的 frame，不过在位图大小较小的情况下，实际操作可能较快。

(2)

**策略 1：哈希表（Hash Table）**

使用哈希表将页号映射到缓冲区中的 frame 索引。每当发出一个读页请求时，可以直接通过哈希表查询该页是否已经存在于缓冲区中。

**时间复杂度：**O(1) 平均时间。
查找哈希表的平均时间复杂度是常数时间，能快速判断页是否在缓冲区中，并返回对应的 frame 索引。

**策略 2：平衡二叉搜索树（例如 AVL 树或红黑树）**

可以使用平衡二叉搜索树来存储页号与 frame 的映射关系。这样可以通过二叉树查找来判断页是否在缓冲区中。

**时间复杂度**：O(log n)，n 是缓冲区中的页数。
通过二叉树查找页号是否存在，并在存在时返回 frame 地址。

2. 我们在讲义上介绍了 SSD 感知的 CF-LRU 算法。即 Clean-First LRU 算法。该算法虽然看起来可以减少对 SSD 的写操作，但依然存在一些问题。请分析一下该算法的主要缺点有哪些？给出两点，并简要解样你的理由

## 1. 写放大现象

尽管 CF-LRU 试图减少对 SSD 的写操作，但在实际应用中，仍可能导致写放大现象。这是因为当一个脏页被替换时，可能会触发对整个块的写入，即使只有一部分数据需要更新。这种情况在高频写入场景下更加明显，导致 SSD 的使用寿命缩短。

## 2. 复杂的管理开销

CF-LRU 需要维护清洁页和脏页的优先级，增加了缓冲区管理的复杂性。在页替换时，算法需要额外判断页的状态，这会导致较高的计算和时间开销，尤其是在页访问模式变化频繁的情况下。这种管理开销可能会抵消其减少写操作的优势，影响系统整体性能。

魏星迪 SA24011277